Data — deep dive

Why this layer

A base model is **frozen at its knowledge cutoff** and has never seen your private data. This layer supplements it with fresh / private knowledge **at answer time — no retraining.** The headline technique is **RAG** (Retrieval-Augmented Generation).

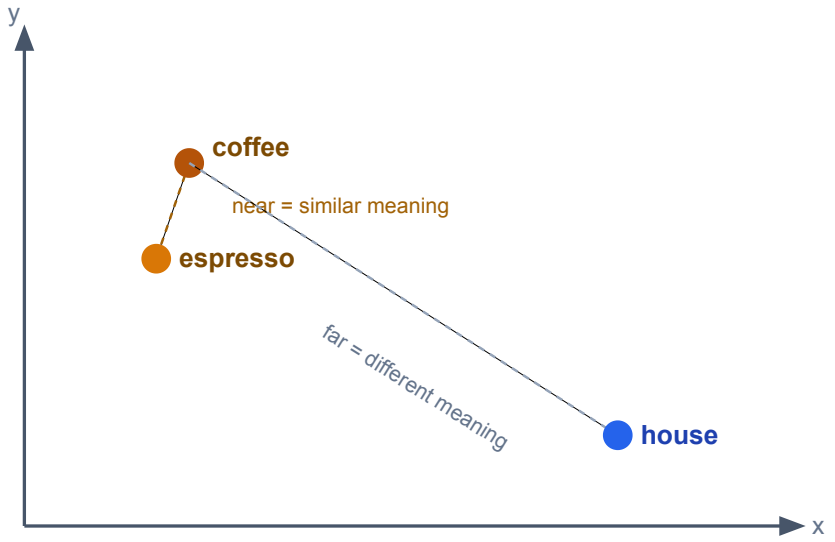
- Need new **knowledge**? → **RAG** (this layer)
- Need new **behaviour**? → **fine-tune** (`02-models/`)

The line that holds the course together: **RAG for knowledge, fine-tune for behaviour.** They stack — fine-tune the tone, retrieve the facts.

How we got here · the evolution of retrieval

Era 1 · Keyword — matches *words*, not meaning. A query for *"my pod keeps restarting"* never finds the doc *"fixing CrashLoopBackOff"* — zero shared words.

Era 2 · Semantic — turn text into a **vector**, so *meaning becomes position*: similar ideas land nearby even with no shared words.



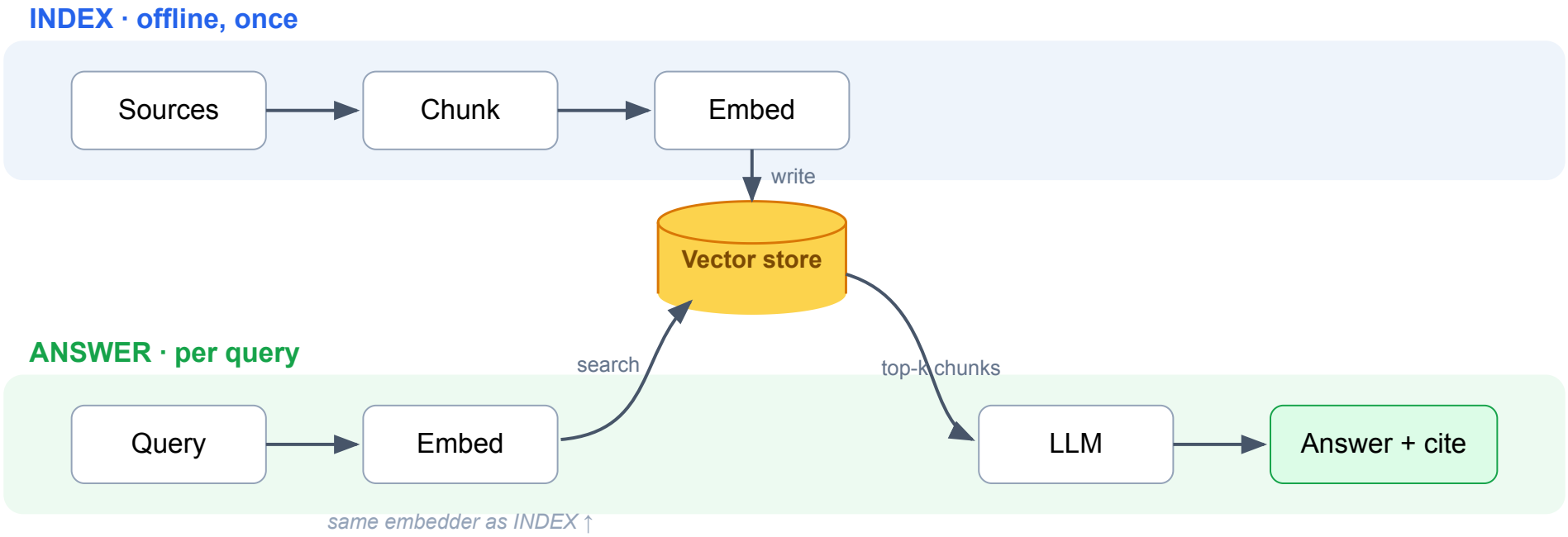
position = meaning · distance = (dis)similarity

RAG vs long context — why not just stuff the window?

Dimension	Long context	RAG
Infrastructure	none — "no-stack stack"	heavy: chunk + embed + vector DB + sync
Reliability	model sees everything	probabilistic → **silent failure**
Cost / query	reprocesses every token, every call	pays once at index time
Data ceiling	~1M tokens	effectively infinite corpus

- **Bounded data + global reasoning** (one contract, one book) → **long context**.
- **Fresh / private / huge** (an enterprise corpus) → **RAG**.

The simple pipeline



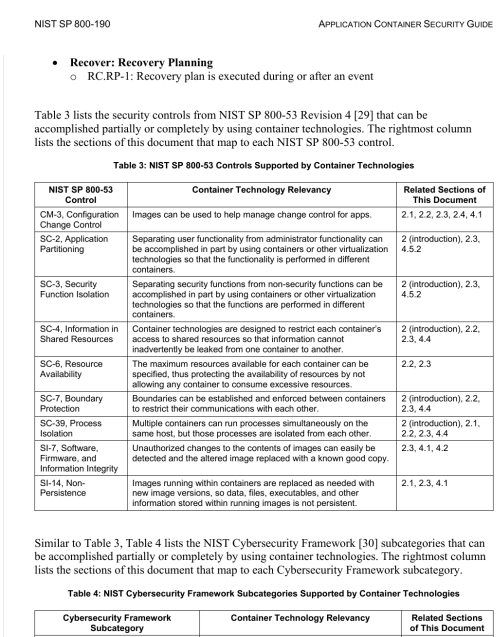
Index once, offline. Per query: embed → fetch **top-k** → prompt → answer.

Naïve on purpose (similarity-only, fixed top-k) — the **advanced pipeline** closes the gaps; **Lab 1** builds it.

Phase 1 · Sources

The raw knowledge. Its **shape** decides how much pipeline work before it's usable.

Shape	Examples	Pipeline cost
Unstructured	PDFs, web, email, transcripts	High — parse, clean, chunk
Semi-structured	Markdown, HTML, JSON, CSV	Medium — structure-aware split
Structured	SQL, APIs, spreadsheets	Low — often query directly



- **Recover: Recovery Planning**
  - RC.RP-1: Recovery plan is executed during or after an event

Table 3 lists the security controls from NIST SP 800-53 Revision 4 [29] that can be accomplished partially or completely by using container technologies. The rightmost column lists the sections of this document that map to each NIST SP 800-53 control.

Table 3: NIST SP 800-53 Controls Supported by Container Technologies

NIST SP 800-53 Control	Container Technology Relevancy	Related Sections of This Document
CM-3, Configuration Change Control	Images can be used to help manage change control for apps.	2.1, 2.2, 2.3, 2.4, 4.1
SC-2, Application Partitioning	Separating user functionality from administrator functionality can be accomplished in part by using containers or other virtualization technologies so that the functionality is performed in different containers.	2 (introduction), 2.3, 4.5.2
SC-3, Security Function Isolation	Separating security functions from non-security functions can be accomplished in part by using containers or other virtualization technologies so that the functions are performed in different containers.	2 (introduction), 2.3, 4.5.2
SC-4, Information in Shared Resources	Container technologies are designed to restrict each container's access to shared resources so that information cannot inadvertently be leaked from one container to another.	2 (introduction), 2.2, 2.3, 4.4
SC-6, Resource Availability	The maximum resources available for each container can be specified, thus protecting the availability of resources by not allowing any container to consume excessive resources.	2.2, 2.3
SC-7, Boundary Protection	Boundaries can be established and enforced between containers to restrict their communications with each other.	2 (introduction), 2.2, 2.3, 4.4
SC-39, Process Isolation	Multiple containers can run processes simultaneously on the same host, but those processes are isolated from each other.	2 (introduction), 2.1, 2.2, 2.3, 4.4
SI-7, Software, Firmware, and Information Integrity	Unauthorized changes to the contents of images can easily be detected and the altered image replaced with a known good copy.	2.3, 4.1, 4.2
SI-14, Non-Persistence	Images running within containers are replaced as needed with new image versions, so data, files, executables, and other information stored within running images is not persistent.	2.1, 2.3, 4.1

Similar to Table 3, Table 4 lists the NIST Cybersecurity Framework [30] subcategories that can be accomplished partially or completely by using container technologies. The rightmost column lists the sections of this document that map to each Cybersecurity Framework subcategory.

Table 4: NIST Cybersecurity Framework Subcategories Supported by Container Technologies

Cybersecurity Framework Subcategory	Container Technology Relevancy	Related Sections of This Document

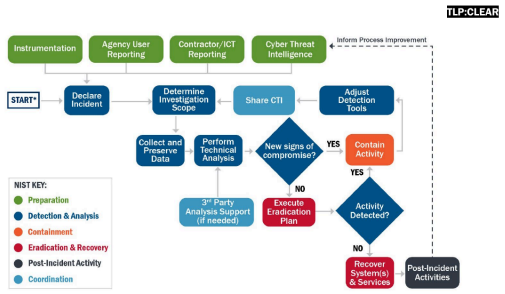


Figure 1: Incident Response Process

Preparation Phase

Prepare for major incidents *before they occur* to mitigate any impact on the organization. Preparation activities include:

- Documenting and understanding policies and procedures for incident response
- Instrumenting the environment to detect suspicious and malicious activity
- Establishing staffing plans
- Educating users on cyber threats and notification procedures
- Leveraging cyber threat intelligence (CTI) to proactively identify potential malicious activity

Define baseline systems and networks before an incident occurs to understand the basics of "normal" activity. Establishing baselines enables defenders to identify deviations. Preparation also includes

- Having infrastructure in place to handle complex incidents, including classified and out-of-band communications
- Developing and testing courses of action (COAs) for containment and eradication
- Establishing means for collecting digital forensics and other data or evidence

The goal of these items is to ensure resilient architectures and systems to maintain critical operations in a compromised state. Active defense measures that employ methods such as redirection and monitoring of adversary activities may also play a role in developing a robust incident response.

Phase 2 · clean & chunk

Clean (strip boilerplate), then **chunk** — passages small enough to embed, large enough to carry meaning. **The highest-leverage, most-overlooked choice in the layer.**

Strategy	When
Fixed-size (N tokens + overlap)	default, robust
Recursive (paragraph→sentence→word)	prose
Structure-aware (headings, code, rows)	docs / code
Semantic (split where topic shifts)	best quality, more compute

Two knobs: **chunk size** (tokens, 256–1024) and **overlap** (~10–20%). Store each chunk with **metadata** (source · page · date · ACL) → filter + cite.

Phase 3 · Embeddings

Text → **vector** that encodes *meaning* — near in meaning, near in space:

```
"how do I reset my password"   → [ 0.21, -0.07, 0.88, … ]
"forgot my login credentials"  → [ 0.19, -0.05, 0.85, … ]
                cosine ≈ 0.97  →  near in meaning  →  retrieve
```

Choose on **dimensions · max input length · domain/language · open vs API**.

> **Non-negotiable rule:** embed documents *and* queries with the **same model**.
> Change the model ⇒ re-embed the entire corpus.

Phase 4 · Vector store

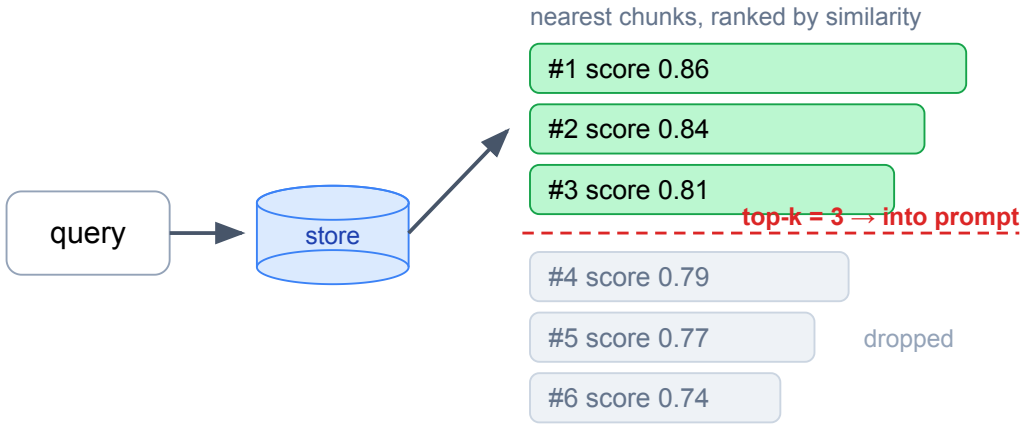
A DB for millions of embeddings, answering *nearest-neighbour* in ms. Real stores use an **ANN** index — trade a sliver of recall for a huge speed-up.

Index	Idea	Trade-off
Flat	compare against every vector	exact, slow past ~100k
HNSW	navigable neighbour graph	fast + high recall; more memory *(default)*
IVF	cluster, search nearest clusters	smaller memory; tune `nprobe`

FAISS *(our labs)* · Chroma · pgvector · Qdrant/Weaviate/Milvus (production scale). Metric must match the embedder (**cosine**/dot). Filter metadata **during** search.

Phase 5 · Retrieval

At answer time the query is embedded with the **same model**; the store returns the **top-k** nearest chunks — and *only* those become the context.



top-k (3–10): too few starves the model; too many buries the answer in noise + cost.

Phase 6 · RAG — putting it together

Stitch the retrieved chunks into the prompt so the model answers **from them**, with a citation back to the source:

```
SYSTEM: Answer only from the CONTEXT. If it isn't there, say you don't know.
CONTEXT: <chunk 1> <chunk 2> … <chunk k>
USER:    <the question>
```

Phase 7 · Evaluation

"The demo answered well" is not evidence. RAG has **two** stages that can each fail:

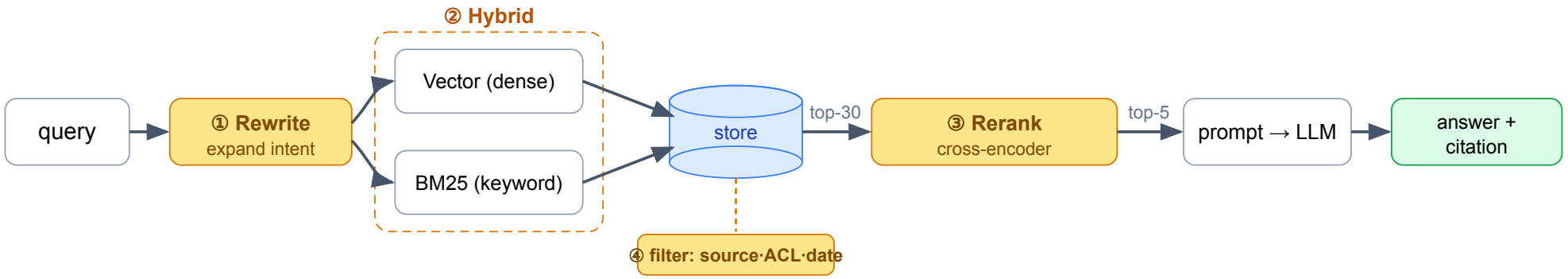
Stage	Question	Metrics
Retrieval	did we fetch the right chunks?	recall@k, precision@k, MRR, hit-rate
Generation	did the answer use them faithfully?	faithfulness, relevance, correctness

Killer failure = **hallucination** (fluent but unsupported). **Faithfulness** measures it. Diagnose by stage: right chunk retrieved but wrong answer → **generation**; right answer impossible → **retrieval**. Tools: **RAGAS** (LLM-as-judge).

RAG on production

Beyond the naïve pipeline — better retrieval, a framework, and operating it at scale.

Advanced pipeline



Four levers (amber) — each closes a gap Chapter 1 left, measured against Phase 7:

- ① **Query understanding** — rewrite / multi-query / HyDE, don't trust the phrasing.
- ② **Hybrid** — dense vectors (meaning) ∪ **BM25** (exact terms: codes, flags, names).
- ③ **Rerank** — over-fetch 30, a cross-encoder keeps the best 5. *Biggest lever after chunking.* · ④ **Metadata filter** — restrict by source · ACL · date.

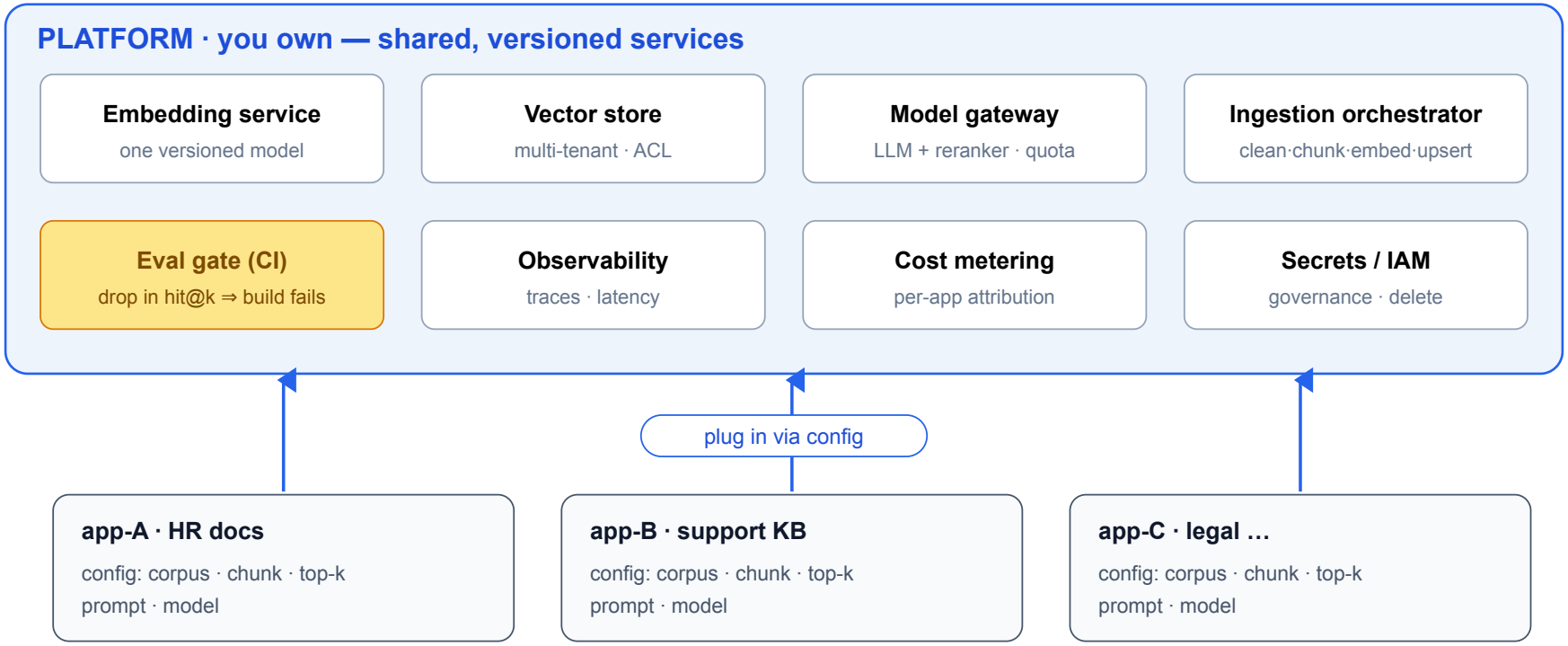
LangChain

A general framework for LLM apps (agents, chatbots, RAG). It adds **no new concept** — it *productizes the seven phases*, one swappable component each.

Phase	By hand	LangChain
1 Sources	`{id,text,meta}` dicts	`Document`
2 Chunk	hand-written splitter	`RecursiveCharacterTextSplitter`
3 Embeddings	`SentenceTransformer.encode`	`HuggingFaceEmbeddings`
4 Vector store	`faiss.IndexFlatIP`	`FAISS.from_documents`
5 Retrieval	hand-written `retrieve()`	`store.as_retriever()`
6 RAG prompt	manual template + `.generate`	`ChatPromptTemplate | chat`

Client-side orchestration; **eval + ops live outside the chain.** Proven in `labs/lab1-langchain/`.

RAG platform



You don't write RAG — you operate a platform; each app is config on top.

Data-ops at fleet scale: freshness · cost · governance (ACL, right-to-delete).

Labs (free Kaggle T4)

Lab	Chapter	What you do
1 · Simple RAG, made visible	1	print the chunks a query loads, top-k, chunk size → grounded + cited Qwen answer
1′ · LangChain build	3	Lab 1 rebuilt component-for-component, same numbers out — the proof
2 · Advanced RAG, real docs	2	clean + chunk real k8s docs, metadata + hybrid + reranker, measure hit@3 / MRR

Same constraints as the Models labs: pick **T4**, fp16 LLM, embedder + reranker on CPU, Internet **On**. Setup in `labs/README.md`.

Takeaways

Retrieval is the heart of RAG — most failures are retrieval failures.

RAG for knowledge, fine-tune for behaviour. Chunking + reranking are the biggest levers.

Measure both stages. A framework writes the chain; **you operate the platform.**